

Resource Allocation System – Iteration 39

Structured Backend Summary Export

Najib Attar

June 2026

1 Iteration Goal

The goal of Iteration 39 was to improve the separation between the backend and frontend by adding structured backend-generated summary files. Iteration 38 introduced `ResourceAllocationSession`, which centralized the backend workflow and made `main.cpp` thinner. However, the Flask dashboard still depended on lower-level CSV outputs and some lifecycle-history text when reconstructing request and allocation summaries.

Iteration 39 addresses this limitation by adding structured summary exports. These exports allow the backend to provide clearer result data that a frontend can read directly in future versions. The change does not replace the old outputs. Instead, it adds new files while preserving compatibility with the existing dashboard and CSV workflow.

2 Changes from Iteration 38

In Iteration 38, `ResourceAllocationSession` became the main backend workflow facade. It coordinated data loading, strategy loading, allocation processing, result exporting, and cleanup.

In Iteration 39, the following changes were added:

- Added `SummaryWriter` as a new backend writer class.
- Added `data/request_summaries.csv` as a structured request-level output.
- Added `data/allocation_summaries.csv` as a structured allocation-level output.
- Updated `ResourceAllocationSession` so summary exports are produced with normal backend exports.
- Updated `main.cpp` so the new summary files are exported and stale summary files are cleared on load failure.
- Updated tests to verify that the new summary files are generated and contain expected fields.

- Lightly updated Flask Raw Data configuration so the new summary files can be viewed.

The existing files `allocations.csv` and `request_results.csv` remain unchanged and are still generated.

3 Scenario

Before Iteration 39, the backend exported allocation rows and request-result rows. These files were useful, but the frontend still had to reconstruct human-friendly summary cards from low-level details. For example, Flask grouped allocation rows by request, read request-result rows, and extracted meeting suggestion text from lifecycle history.

After Iteration 39, the backend also exports structured summary files:

- `request_summaries.csv` contains one row per request.
- `allocation_summaries.csv` contains one row per allocation.

This makes it easier for a frontend to display request and allocation summaries without manually rebuilding all of the summary information from raw backend outputs.

4 Modules and Their Tasks

Module	Task
<code>SummaryWriter</code>	Writes structured request summaries and allocation summaries to CSV files.
<code>ResourceAllocationSession</code>	Coordinates the backend workflow and now calls <code>SummaryWriter</code> after the normal result exports.
<code>DataController</code>	Continues to load data and export the original allocation and request-result files.
<code>AllocationService</code>	Continues to process requests using the selected strategy and rule pipeline.
<code>main.cpp</code>	Remains thin and delegates workflow to <code>ResourceAllocationSession</code> . It now reports that summary files were exported.
<code>web/app.py</code>	Lightly updated so the new summary files are visible through the Raw Data page. It does not compute backend summaries.

Table 1: Main modules involved in Iteration 39

5 Request Summary Export

The new `request_summaries.csv` file provides one structured row per request. It is intended to give frontends a cleaner request-level result view.

The exported fields include:

- request ID
- request type
- title
- purpose
- requester name
- requester role
- priority
- status
- rejection reason
- required participants
- assigned participants
- allocation count
- allocated spaces
- required feature
- required building
- summary text

The `summaryText` field provides a concise backend-generated explanation, such as an approved request with its allocation count or a rejected request with its rejection reason.

6 Allocation Summary Export

The new `allocation_summaries.csv` file provides one structured row per allocation. It combines allocation details with request details so a frontend does not need to manually join request and allocation information.

The exported fields include:

- allocation ID

- request ID
- request title
- request purpose
- request type
- space name
- space type
- building
- day
- start time
- end time
- assigned participants
- space capacity

This supports clearer schedule and allocation-summary displays in future frontend updates.

7 Meeting Suggestion Export Decision

A separate `meeting_suggestions.csv` file was not added in this iteration. Committee meeting suggestions are currently appended to request lifecycle history as text after the suggestion service runs. Exporting them as structured rows would require either parsing the lifecycle-history text or changing how suggestions are stored after generation.

This was intentionally left as future work. Iteration 39 focuses on structured request and allocation summaries without changing the suggestion storage model.

8 Iteration Comparison

Area	Iteration 38	Iteration 39
Backend workflow	Centralized through <code>ResourceAllocationSession.structured</code>	Same workflow, now with summary exports.
Original outputs	<code>allocations.csv</code> and <code>request_results.csv</code> .	Preserved unchanged.
Structured summaries	Not available.	Added request and allocation summary CSV files.
Frontend coupling	Flask still reconstructs summaries from lower-level outputs.	Backend now provides structured files that can reduce this coupling in future work.
Meeting suggestions	Stored in lifecycle history text.	Still stored in lifecycle history text; no separate structured suggestion file yet.
Algorithms and rules	Unchanged.	Unchanged.

Table 2: Comparison between Iteration 38 and Iteration 39

9 Updated Class Diagram

The updated class diagram should show the new `SummaryWriter` class and its relationship with the backend workflow.

The diagram should include:

- `ResourceAllocationSession`
- `SummaryWriter`
- `DataController`
- `AllocationService`
- `Request`
- `Allocation`
- `SystemData`

It should show that `ResourceAllocationSession` uses `SummaryWriter` after allocation processing. `SummaryWriter` reads request and allocation data and writes structured summary files.

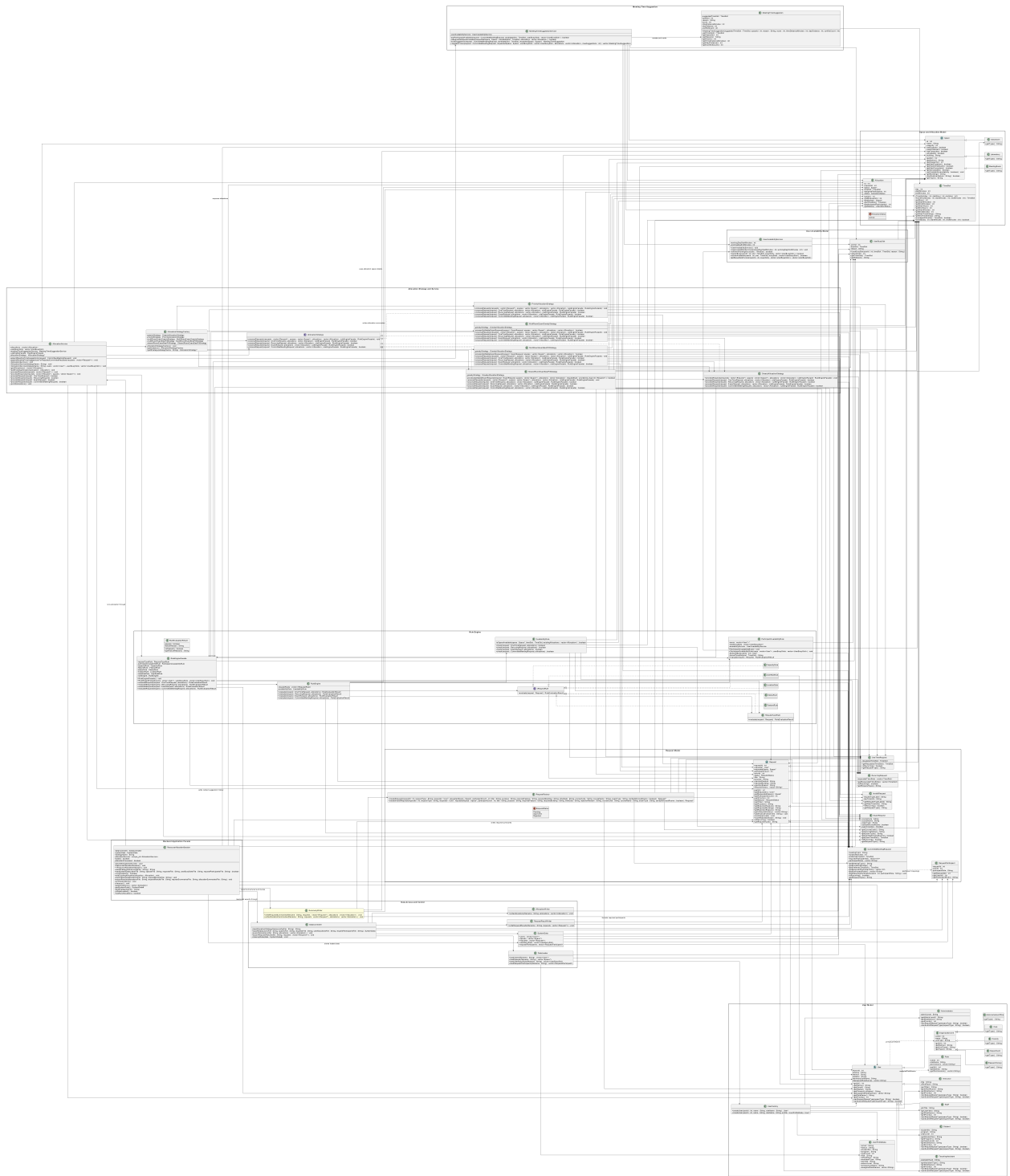


Figure 1: Updated class diagram for Iteration 39

10 Updated Workflow Diagram

The updated workflow diagram should show the backend workflow from startup to export.

The workflow should include:

1. Backend starts through `main.cpp`.
2. `ResourceAllocationSession` loads the selected strategy.
3. `ResourceAllocationSession` loads system data.
4. `AllocationService` processes requests using the selected strategy.
5. Original outputs are exported: `allocations.csv` and `request_results.csv`.
6. Structured summaries are exported: `request_summaries.csv` and `allocation_summaries.csv`.
7. Flask Raw Data can display the new summary files.



Figure 2: Updated workflow diagram for Iteration 39

11 Prototype Implementation

The implementation added the following backend files:

- `src/data/SummaryWriter.h`
- `src/data/SummaryWriter.cpp`

The implementation updated:

- `src/services/ResourceAllocationSession.h`
- `src/services/ResourceAllocationSession.cpp`
- `src/main.cpp`
- `tests/test_resource_allocation_session.cpp`
- `tests/test_integration.cpp`
- `web/app.py`

The backend test command used was:

```
powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1
```

The test result was:

58 passed, 0 failed

Backend smoke testing passed. Flask smoke testing also passed for the following routes:

- `/`
- `/allocation-summary`
- `/request/40`
- `/raw-data?file=request_summaries.csv`
- `/raw-data?file=allocation_summaries.csv`

12 What Did Not Change

Iteration 39 intentionally avoided changing backend behavior. The following parts remain unchanged:

- allocation algorithms
- rule behavior
- strategy selection behavior
- request factory behavior
- user factory behavior
- committee meeting approval and rejection behavior
- exam sharing behavior
- existing output files
- CSV input formats
- main Flask summary pages

The new summary files are additional outputs, not replacements.

13 Remaining Limitation and Future Work

This iteration improves backend output structure, but it does not completely remove frontend coupling. Flask can now access `request_summaries.csv` and `allocation_summaries.csv` through Raw Data, but some human-friendly pages may still build cards from existing CSVs and lifecycle-history text.

Future work may include:

- updating Flask summary pages to use the new structured summary files directly
- exporting meeting suggestions as structured rows
- storing meeting suggestions in a structured backend model after generation
- reducing frontend dependence on lifecycle-history text
- adding a stable API layer on top of `ResourceAllocationSession`

14 Summary

Iteration 39 added structured backend summary exports through a new `SummaryWriter`. The backend now produces both request-level and allocation-level summary files in addition to the existing output files. This improves frontend independence by giving future UIs clearer backend-owned result data.

The change is incremental and safe. It preserves existing behavior, keeps old outputs for compatibility, and does not alter allocation algorithms, rules, factories, strategies, or request processing. It prepares the project for future frontend cleanup where Flask or another UI can use structured summary files instead of reconstructing summaries manually.